Session 4: Matplotlib & Data Visualization

ESS: Programming with Python

Abate Alemayehu

Ethiopian Statistical Service (ESS)

July 20–28, 2026

Matplotlib Overview

Sessions

- Session 1: Matplotlib Basics (Slides 256–270)
- Session 2: Introduction to Data Visualization (Slides 271–285)
- Session 3: Customizing Plots and Adding Interactivity (Slides 286–305)
- Session 4: Advanced Plotting Techniques (Slides 306–325)
- Session 5: Best Practices for Data Visualization (Slides 326–340)

Info

"A picture is worth a thousand words." Learn to create publication-quality figures.

Slide 256: What is Matplotlib?

Key Facts

- Python's primary plotting library
- Created by John Hunter (2003)
- MATLAB-style interface
- High-quality publication-ready figures

Info

Matplotlib is the foundation of visualization in the Python scientific stack.

Slide 257: Installing and Importing

Installation

- Included in Anaconda
- `conda install matplotlib` or `pip install matplotlib`

Standard Import

```
1 import matplotlib.pyplot as plt
```

Slide 258: Jupyter Magic Commands

Magic Commands

- `%matplotlib inline` → plots appear in notebook
- `%matplotlib notebook` → interactive plots
- Run before plotting

```
%matplotlib inline
import matplotlib.pyplot as plt
```

Slide 259: Basic Line Plot

```
1 import matplotlib.pyplot as plt
2
3 plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
4 plt.show()
```

Slide 260: Figure and Axes

Concepts

- **Figure:** entire window (container)
- **Axes:** actual plot area (can have multiple)
- `fig, ax = plt.subplots()` → create both

Info

The object-oriented interface (`fig, ax`) gives more control.

Slide 261: Adding Labels and Title

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
plt.xlabel('X axis label')
plt.ylabel('Y axis label')
plt.title('Plot Title')
plt.show()
```

Slide 262: Adding Legend

```
plt.plot(x, y, label='Data')
plt.legend()
plt.legend(loc='upper left')
```

Legend Locations

'upper left', 'upper right', 'lower left', 'lower right', 'best'

Slide 263: Customizing Line Style

```
plt.plot(x, y, linestyle='--')
plt.plot(x, y, linewidth=2)
plt.plot(x, y, color='red')
plt.plot(x, y, color='#FF0000')
plt.plot(x, y, marker='o')
```

Slide 264: Line Style Shorthand

```
'r--'    # red dashed
'g*'     # green star markers
'b-'     # blue solid line
'ro'     # red circle markers (no line)
```

Format String: [color] [marker] [line]

Examples: 'r-', 'gs:', 'b*-'

Slide 265: Scatter Plot

```
plt.scatter(x, y)
plt.scatter(x, y, c=colors, s=sizes, alpha=0.5)
```

Info

Scatter plots are ideal for showing relationships between two variables.

Slide 266: Grid and Axis Limits

```
plt.grid(True)
plt.xlim(0, 10)
plt.ylim(0, 100)
plt.axis([0, 10, 0, 100])
```

Slide 267: Tick Customization

```
plt.xticks([0, 1, 2, 3])
plt.xticks(rotation=45)
plt.yticks(fontsize=12)
```

Slide 268: Saving Figures

```
1 plt.savefig('filename.png')
2 plt.savefig('filename.png', dpi=300)
3 plt.savefig('filename.png', bbox_inches='tight')
4 plt.savefig('filename.png', transparent=True)
```

Info

Use `.pdf` or `.svg` for vector graphics (publication quality).

Slide 269: Multiple Plots with subplot()

```
plt.subplot(2, 2, 1)
plt.plot(x, y1)

plt.subplot(2, 2, 2)
plt.plot(x, y2)

# Using subplots()
fig, axes = plt.subplots(2, 2)
axes[0, 0].plot(x, y1)
```

Slide 270: Code Example: Basic Plot

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, 'b-', label='sin(x)', linewidth=2)
plt.plot(x, y2, 'r--', label='cos(x)', linewidth=2)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Sine and Cosine Functions')
plt.legend()
plt.grid(True)
plt.savefig('sine_cosine.png', dpi=150)
plt.show()
```

Slide 271: Why Visualize Data?

- Humans process visuals faster than numbers
- Reveal patterns, trends, outliers
- Communicate insights effectively
- Exploratory data analysis (EDA)

Slide 272: Types of Data

Numerical

- Continuous (height, temperature)
- Discrete (count, number of children)

Categorical

- Nominal (color, city name)
- Ordinal (size: small, medium, large)

Time Series

Data over time (stock prices, temperature trends)

Slide 273: Choosing the Right Plot

Goal	Recommended Plot
Trend over time	Line plot
Compare categories	Bar chart
Distribution	Histogram, Box plot
Relationship	Scatter plot
Composition	Stacked bar, Pie chart

Slide 274: Line Plots – Best Practices

- Use for time series or continuous data
- Multiple lines should be distinguishable
- Label axes clearly
- Don't over-plot too many lines

Slide 275: Bar Charts – Best Practices

- Use for comparing categories
- Start y-axis at zero
- Sort bars meaningfully
- Horizontal bars for many categories

Slide 276: Histograms – Best Practices

- Shows distribution of single variable
- Choose bin size carefully
- Too few bins $\rightarrow$ oversimplified
- Too many bins $\rightarrow$ noisy

Info

Rule of thumb: `bins = int(np.sqrt(N))` where N is number of data points.

Slide 277: Box Plots – Best Practices

- Shows median, quartiles, outliers
- Good for comparing distributions
- Box = IQR (25th to 75th percentile)
- Whiskers = $1.5\times$ IQR
- Points beyond whiskers = outliers

Slide 278: Scatter Plots – Best Practices

- Shows relationship between two variables
- Add transparency (alpha) for over-plotting
- Color by third variable
- Size by fourth variable

Slide 279: Pie Charts – Cautions

- Hard to compare angles
- Use only for 2-3 categories
- Consider bar charts instead
- Don't use 3D pie charts

Warning

Pie charts are often misused. Bar charts are almost always better.

Slide 280: Avoiding Common Mistakes

- Truncated y-axis (exaggerates differences)
- Missing labels/titles
- Too much ink (chart junk)
- Wrong plot type for data

Slide 281: Color Choices

- **Sequential** (light to dark) for ordered data
- **Diverging** for data with critical midpoint
- **Qualitative** for categories
- Colorblind-friendly palettes: `'viridis'`, `'plasma'`

Slide 282: Accessibility

- Avoid red-green (common colorblindness)
- Use patterns or labels in addition to color
- Sufficient contrast
- Readable font sizes ($\geq$10pt for publication)

Slide 283: Exploratory vs Explanatory

Exploratory

- Many plots, messy
- For yourself
- Quick and iterative

Explanatory

- Polished, clear
- For audience
- Focused message

Slide 284: The Data-ink Ratio (Tufte)

Principle

- Maximize ink showing data
- Minimize non-data ink (gridlines, borders)
- Remove redundant elements
- Simplify without losing information

Info

"Above all else show the data." – Edward Tufte

Slide 285: Code Example: Multiple Plot Types

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
data = np.random.normal(0, 1, 1000)
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 12, 67]

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Histogram
axes[0,0].hist(data, bins=30, edgecolor='black')
axes[0,0].set_title('Histogram')

# Box plot
axes[0,1].boxplot([data, data+2, data-1])
```

Slide 286: Figure Size and DPI

```
plt.figure(figsize=(8, 6), dpi=100)
```

- `figsize` = (width, height) in inches
- `dpi` = dots per inch (resolution)
- Set before plotting

Slide 287: Colors in Matplotlib

- Named colors: `'red'`, `'blue'`, `'green'`, `'orange'`
- Hex codes: `'#FF5733'`
- RGB tuples: `(0.2, 0.4, 0.6)`
- Grayscale: `'0.5'` (50% gray)

Slide 288: Colormaps

- `plt.colormaps()` → list all colormaps
- **Sequential:** `'viridis'`, `'plasma'`, `'inferno'`
- **Diverging:** `'coolwarm'`, `'RdBu'`
- **Qualitative:** `'Set1'`, `'tab10'`

Slide 289: Using Colormaps

```
plt.scatter(x, y, c=values, cmap='viridis')
plt.colorbar()

plt.imshow(matrix, cmap='hot')
```

Slide 290: Customizing Fonts

```
# Global settings
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 12

# Per element
plt.title('Title', fontsize=14, fontweight='bold')
```

Slide 291: Annotations and Text

```
plt.text(x, y, 'text')

plt.annotate('label',
             xy=(x,y),
             xytext=(x_offset,y_offset),
             arrowprops=dict(arrowstyle='->'))
```

Slide 292: Adding Shapes

```
plt.axhline(y=0, color='k', linestyle='--')
plt.axvline(x=0, color='k')
plt.axspan(xmin, xmax, alpha=0.2)
```

Slide 293: Subplots with subplots()

```
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(12, 8))
axes[0,1].plot(...)
fig.tight_layout()
```

Slide 294: Shared Axes

```
plt.subplots(2, 2, sharex=True, sharey=True)
plt.subplots(2, 2, sharex='col')
```

Slide 295: Adjusting Subplot Spacing

```
plt.subplots_adjust(hspace=0.3, wspace=0.3)
fig.tight_layout(pad=2.0)
```

Slide 296: Twin Axes (Two y-scales)

```
ax1 = plt.gca()
ax2 = ax1.twinx()
ax1.plot(x, y1, 'b-')
ax2.plot(x, y2, 'r-')
```

Slide 297: Logarithmic Scales

```
plt.xscale('log')
plt.yscale('log')
plt.loglog(x, y)
```

Slide 298: Error Bars

```
1 plt.errorbar(x, y, yerr=y_err, fmt='o')
2 plt.errorbar(x, y, xerr=x_err, fmt='o')
```

Slide 299: Filling Between Curves

```
plt.fill_between(x, y1, y2, alpha=0.3)
plt.fill_between(x, y, where=(y > 0),
                 color='green', alpha=0.3)
```

Slide 300: Interactive Plotting in Jupyter

```
%matplotlib notebook
%matplotlib inline
```

Slide 301: Event Handling (Advanced)

- `plt.gcf().canvas.mpl_connect('button_press_event', on_click)`
- Custom interactive behavior
- Beyond scope of intro workshop

Slide 302: Saving Interactive Plots

- Save as HTML with `mpld3` library
- `mpld3.save_html(fig, 'plot.html')`
- Interactive plots in browser

Slide 303: Style Sheets

```
plt.style.available
plt.style.use('ggplot')
plt.style.use('seaborn-v0_8')
plt.style.use('dark_background')
```

Slide 304: Custom Style Sheets

- Create your own `.mplstyle` file
- Consistent look across all plots
- Share with team

Slide 305: Code Example: Customized Plot

```python
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('seaborn-v0_8-darkgrid')

x = np.linspace(0, 10, 50)
y = np.sin(x)
y_err = 0.1 + 0.1 * x

fig, ax = plt.subplots(figsize=(10, 6))

ax.errorbar(x, y, yerr=y_err, fmt='o-', capsize=3,
            color='#2E86AB', ecolor='#A23B72',
            label='sin(x) with error')

ax.fill_between(x, y - y_err, y + y_err,
```

Slide 306: 3D Plotting – Setup

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
```

Slide 307: 3D Scatter Plot

```
1 ax.scatter3D(x, y, z, c=z, cmap='viridis')
```

Slide 308: 3D Surface Plot

```
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
ax.plot_surface(X, Y, Z, cmap='viridis')
ax.contour3D(X, Y, Z, 50, cmap='binary')
```

Slide 309: 3D Wireframe Plot

```
1 ax.plot_wireframe(X, Y, Z, color='black')
```

Slide 310: Contour Plots

```
plt.contour(X, Y, Z, levels=20)
plt.contourf(X, Y, Z, levels=20)
plt.colorbar()
```

Slide 311: Density Plots

```
plt.hist2d(x, y, bins=40, cmap='Blues')
plt.colorbar()
plt.hexbin(x, y, gridsize=30)
```

Slide 312: Violin Plots

```
1 plt.violinplot([data1, data2, data3])
```

Slide 313: Heatmaps

```
plt.imshow(matrix, cmap='hot', aspect='auto')
plt.colorbar()
plt.xticks(ticks, labels)
```

Slide 314: Stack Plots

```
plt.stackplot(x, y1, y2, y3,
              labels=['A','B','C'])
```

Slide 315: Step Plots

```
1 plt.step(x, y, where='mid')
```

Slide 316: Stem Plots

```
plt.stem(x, y, linefmt='b-',
         markerfmt='bo', basefmt='r-')
```

Slide 317: Quiver Plots (Vector Fields)

```
plt.quiver(x, y, u, v, scale=30)
```

Slide 318: Streamplots

```
1 plt.streamplot(x, y, u, v, density=1)
```

Slide 319: Geographic Mapping with Basemap (Legacy)

- Basemap is deprecated
- Use **Cartopy** instead
- `import cartopy.crs as ccrs`
- `ax = plt.axes(projection=ccrs.PlateCarree())`

Slide 320: Cartopy Example

```
import cartopy.crs as ccrs
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(1, 1, 1, projection=ccrs.Robinson())
ax.stock_img()
ax.coastlines()
plt.title('World Map')
plt.show()
```

Slide 321: Animations with FuncAnimation

- `from matplotlib.animation import FuncAnimation`
- Define update function
- `anim = FuncAnimation(fig, update, frames=100, interval=50)`
- Save: `anim.save('animation.mp4')`

Slide 322: Animation Example Skeleton

```
def update(frame):
    ax.clear()
    ax.plot(x[:frame], y[:frame])
    return ax

anim = FuncAnimation(fig, update,
                     frames=len(x), interval=20)
```

Slide 323: Handling Large Datasets

- `plt.hexbin` for 2D density (fast)
- Downsample: `plt.plot(x[::10], y[::10])`
- Use `rasterized=True` for vector formats
- `plt.scatter` with `alpha` and small `s`

Slide 324: Combining with Pandas

```
df.plot(kind='scatter', x='x', y='y', ax=ax)
df.groupby('category').plot(x='time',
                            y='value',
                            ax=ax)
```

Slide 325: Code Example: Advanced Plot

```
import matplotlib.pyplot as plt
import numpy as np

fig = plt.figure(figsize=(14, 6))

# 3D Surface
ax1 = fig.add_subplot(121, projection='3d')
x = np.linspace(-3, 3, 50)
y = np.linspace(-3, 3, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))
ax1.plot_surface(X, Y, Z, cmap='coolwarm')
ax1.set_title('3D Surface')

# Contour + Heatmap
ax2 = fig.add_subplot(122)
```

Slide 326: Know Your Audience

- Technical audience $\rightarrow$ more detail
- General audience $\rightarrow$ simpler, clearer
- Executive $\rightarrow$ key insights only
- Adjust complexity accordingly

Slide 327: Start with a Question

- What am I trying to show?
- What is the main message?
- Every element should support that message
- Remove anything that doesn't

Slide 328: Title and Labels Best Practices

- Title: clear, descriptive, concise
- Axes labels: include units
- Legend: only if needed (use direct labels when possible)
- No "Plot of..." in title (obvious)

Slide 329: Axes and Scale Best Practices

- Start y-axis at zero for bar charts
- Log scales: clearly indicate
- Consistent scales for comparison
- Don't truncate axes (misleading)

Slide 330: Color Best Practices

- Use color intentionally
- Avoid red-green combinations
- Use sequential colormaps for ordered data
- Use diverging for deviations from midpoint
- Check on grayscale (print compatibility)

Slide 331: Accessible Color Palettes

- `'viridis'` (perceptually uniform)
- `'cividis'` (colorblind-friendly)
- `'plasma'`, `'inferno'`, `'magma'`

Slide 332: Font and Text Best Practices

- Readable font size ($\geq$10pt)
- Sans-serif fonts for digital
- Serif fonts for print
- Consistent font family

Slide 333: Avoiding Chart Junk

- No unnecessary 3D effects
- No unnecessary gridlines
- No distracting backgrounds
- No redundant labels
- **Simplify, simplify, simplify**

Slide 334: Telling a Story with Data

- Guide the viewer's eye
- Use annotations to highlight key points
- Order data meaningfully (e.g., sorted bars)
- Add trend lines or highlights

Slide 335: Multi-panel Figures

- Consistent axes across panels
- Label panels (A, B, C)
- Same scale for comparison
- Use shared axes when possible

Slide 336: Publication Quality Figures

- Vector formats (PDF, SVG) for scaling
- 300 DPI for raster (PNG, JPG)
- Match journal requirements
- Include scale bars where needed

Slide 337: Common Pitfalls to Avoid

- Pie charts with many slices
- 3D bar charts (hard to read)
- Dual axes without clear labeling
- Over-plotting (too many points)
- Missing error bars

Slide 338: Checking Your Plot Checklist

Checklist

- Title describes message
- Axes labeled with units
- Legend clear (or not needed)
- Colors accessible
- No misleading truncation
- Fonts readable
- Message is clear without text

Slide 339: Tools for Creating Visualizations

- **Matplotlib** (this workshop)
- **Seaborn** (statistical plots, easier defaults)
- **Plotly** (interactive, web-based)
- **Altair** (declarative, interactive)
- **ggplot** (R-like syntax)

Slide 340: Summary: Principles of Good Visualization

1. Show the data clearly
2. Reduce clutter
3. Use appropriate scales
4. Label everything
5. Tell a story
6. Be honest
7. Consider your audience

End of Session 4

Questions?

Next: Session 5 – SciPy & Image Processing